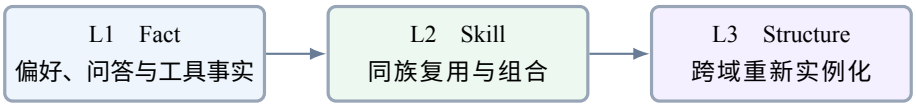
Tri-EvoBench

Agent 自进化 Benchmark 实施方案

Hierarchical Learner – Streaming Tasks – Interactive Environment

评测 learner 如何逐轮从交互轨迹中更新，并形成递进的可复用能力



正文约 5 页 + 构造来源与竞品 Appendix

真实场景抽象：episode 内多轮用户–Agent–环境交互；episode 间 learner 更新。

1 动机：为什么需要多层级、Online 的自进化 Benchmark

Agent 自进化方法已经能够更新 memory、skill、harness 或模型参数，但现有评测往往只覆盖单一载体、单一领域或很短的任务序列，并主要比较演进前后的终点分数：同样的增益可能只是记住一条事实，也可能来自可复用 procedure 或跨任务族结构，因而无法说明 learner 究竟形成了哪一层能力；终点指标也会掩盖早期错误、错误经验累积和遗忘。Tri-EvoBench 因此聚焦评测 **learner 在真实流式任务场景中的持续演进能力**：任务不断到达、反馈逐轮揭示、环境状态持续变化，learner 必须边执行、边吸收经验、边更新自身，并让收益稳定累积。Benchmark 在同一 online loop 中依次测量事实更新、任务级能力复用和跨任务族结构迁移；每条流持续 100+ episodes，信息由用户逐步披露，失败后只返回局部不足。

核心主张

Benchmark 的基本实验单位不是独立题目，而是一个带持久状态的交互任务流。每个 episode 内，用户 simulator、task Agent 与工具环境进行多轮交互；episode 结束后，learner 才读取轨迹、环境反馈和已有状态并产生一次更新。对相同初始 snapshot 比较 Reset、Persistent、Unrelated-control 与 Oracle-update，从而回答“learner 从哪段经验学到了什么、更新处于哪一抽象层、能否在延迟任务中触发，以及是否破坏旧能力”。

1.1 与现有 benchmark 的区别

Benchmark / 协议	主要评测对象	层级	流式	模拟用户	混合任务族	主要区别
StreamBench / LifelongAgentBench	连续改进、技能依赖	否	是	否	部分	通常不把事实、技能、跨族结构拆成递进层级。
Evo-Memory / EvoMemBench	记忆更新、保留与调用	部分	是	否	部分	聚焦 memory lifecycle，而非跨层级能力跃迁。
SEA-Eval / BenchTrace	重复、干扰、失败模式	否	是	否	部分	精细诊断效率或反思，但 L3 迁移不是核心。
AgentStream	多域共享状态	否	是	部分	是	改变流结构，缺少带构造真值的任务关系。
SkillLearnBench / SkillsBench	skill 生成或 skill 边际收益	部分	部分	否	是	多数任务预先绑定 skill；主要测生成/使用，不测完整层级。
Harness-evolution protocols	harness 更新与受益	否	是	否	是	强调预算和 sealed test，不规定学习内容的能力层级。
Tri-EvoBench（拟）	**轨迹驱动的 evolution learner**	**是**	**是**	**是**	**是**	**按 learner 更新抽象层次组织能力图、混合流与因果对照。**

表 1 设计差异。“模拟用户”指任务信息由独立 user process 按交互逐步披露；源数据偶尔含对话并不等价于统一的用户模拟协议。

1.2 Benchmark 的三个核心特征

1. **逐轮计分的在线演进。**每个 episode 都计入最终结果，而不是只比较任务流结束后的 final checkpoint。任务连续到达，learner 在第 t 轮只能利用此前轨迹更新状态，再接受第 $t{+}1$ 轮测试；最终同时报告逐轮平均分、学习曲线和旧任务 replay，使早期错误、收敛速度与遗忘都无法被最后几轮的提升掩盖。
2. **真实的用户–Assistant 交互与局部反馈。**用户不会一次性给出完整事实或 gold procedure，而是在 Agent 澄清时逐步披露信息；执行失败后，反馈只指出当前回答或行动的不足，例如遗漏约束、违反偏好或工具前置条件未满足，不直接提供标准答案或应写入的 skill。Learner 必须从有限反馈与完整轨迹中定位失败原因，并决定是否、以及如何更新。
3. **多层级的演化任务。**同一评测框架依次测量事实更新、同族任务能力复用和跨任务族结构迁移。三级任务共享 online interaction / update loop，却通过不同的 probe 与对照区分“记住一条信息”“学会一类任务”和“抽象出可迁移结构”，从而刻画现有自进化方法的能力边界。

2 问题形式化：两层时间尺度上的 Evolution Learner

问题显式区分 task Agent A 与 self-evolution learner L。设第 t 个 episode 的用户、环境、请求、完整轨迹、局部反馈、得分和持久状态分别为 $U_t, E_t, x_t, \tau_t, f_t, r_t, s_t$：

$$y_t, \tau_t, r_t = \text{Interact}(A(M, s_t), U_t, E_t; x_t), \qquad s_{t+1} = L(s_t, \tau_t, f_t).$$

内层 **interaction loop** 允许用户补充信息、Agent 澄清并调用工具；外层 **evolution loop** 每个 episode 只更新一次，输入必须包含完整轨迹而不只是最终答案。反馈 f_t 只描述可观察到的不足，不包含 gold answer / gold skill。Benchmark 冻结底座模型、工具权限、token / rollout 预算与 verifier，只允许被声明的 Context、Harness 或 Parameter state 跨 episode 保留。主分数采用 $T^{-1}\sum_{t=1}^{T} r_t$，并报告整条 learning curve；隐藏 verifier 的完整结果不写回 learner，避免评分信息反过来泄露解法。

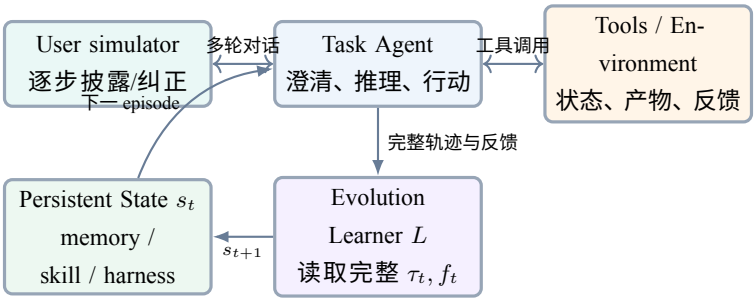


图 1 两层时间尺度：episode 内多轮交互，episode 间 learner 从轨迹更新。

2.1 一条流中必须有不同任务，而非重复单一模板

每条流按配额混合目标能力的 exposure / probe、共享表面但能力不同的 near-negative、完全无关任务和旧能力 replay。L1 交错偏好任务、事实问答、工具调用与普通 distractor；L2 在一条流中交错相关 task families、surface-shift 与无关 family；L3 的 source 和 target 必须来自不同 domain、representation、tool space 与 verifier。生成器同时改变实体、模态、产物与语言表面，并限制任一模板不超过单条流的 30%。Hierarchy 描述的是 learner 应从这些轨迹中形成的更新粒度，不是把任务按 easy / medium / hard 分箱。

2.2 多轮交互：用户不是把全部信息一次性写进 prompt

每个任务有一个独立、冻结的 user simulator state，包含目标、已知事实、偏好、可透露字段和纠正事件。Agent 必须主动澄清缺失的高价值 slot；用户只回答被问到的内容，可能在后续轮次补充或修正信息。Core 同时提供：

- **Disclosed arm**：必要信息随初始请求直接给出，用来测纯执行上限；
- **Interactive arm**：隐藏 1–3 个必要字段，按真实风格和任务约束逐步透露；
- **Correction arm**：用户在执行前更新一个事实或偏好，测覆盖旧状态而非叠加冲突。

User simulator 不自行发明环境事实，所有回答由 profile / world state grounding；风格可由 RealUserSim profiles 采样，任务交互骨架借鉴 τ-bench 与 AppWorld。最终 reward 只由环境状态和确定性 verifier 决定，用户 LLM 不担任裁判。相关 benchmark 的原始构造见 Appendix A。

2.3 共同对照与输出

主结果至少包含 Reset/Frozen、Raw-Replay、Evolving、Matched-Search 和 Oracle-Update。除 task success 外，报告 update precision、activation/adherence、forward transfer、replay regression、stale / intrusion error、token / tool / wall-clock cost，并保存每轮 state diff。这样能够把“不会写更新”和“有更新但不会使用”分开。

3 L1：事实性在线更新（FactStream）

L1 只测最基础的事实学习：learner 能否从多轮用户–Agent–工具轨迹中识别并保存用户偏好、个人/工作事实和工具所需事实，在后续问答、内容生成或工具调用中正确使用；当用户纠正信息时，能够用新事实覆盖旧事实。L1 不引入复杂 policy、审批流程或规则自动机，避免把事实记忆与程序归纳混在一起。

技术路线：版本化 key–value facts + 多种任务 renderer + grounded user simulator

每条事实只有 scope、value、observed time、valid time 与 supersedes 等最小字段。Generator 先采样事实及其未来使用位置，再把它渲染到自然对话、任务附件或工具返回中；learner 只看到完整轨迹，不看到结构化 fact record。问答用规范化答案评分，偏好任务检查产物属性，工具任务检查调用参数与最终状态。复杂 procedure 留给 L2。

3.1 三类事实、三类任务

事实类型	例子	任务形式	Verifier
用户偏好	默认语言、格式、文件命名、通知方式、会议时段	写邮件/文档、生成摘要、安排日程	产物属性与日历状态
个人/工作事实	联系人别名、项目 owner、地点、截止时间、当前版本	延迟问答、带事实的内容生成	规范化答案与字段检查
工具事实	默认 calendar、账号、目录、timezone、联系人 ID	calendar / file / message mock tools	工具名、arguments 与终态

表 2 L1 只改变事实的内容、作用域和时间，不要求学习复杂 procedure。

一条 Core 流固定为 120 episodes：24 个事实披露/修正，24 个偏好任务，24 个事实问答，24 个依赖事实的工具调用，16 个无关 distractor / negative task，以及 8 个长延迟 replay。事实首次出现与 probe 的间隔在 5–80 episodes 之间；12 次修正分布在流的中后段；每 10 个 episode 至少混合三种任务形式。Interactive arm 将部分事实放在 Agent 澄清后的用户回复中，Disclosed arm 则在初始请求直接给出，用于分离交互获取与长期保存。

3.2 可从哪些 benchmark / 数据中提取

- **ClawArena** [16] 的 staged updates、隐式偏好和持久 workspace 最接近“事实随时间到达并被后续任务使用”的流式结构；可抽取更新位置、冲突类型和延迟 probe，而不复用其复杂专业问题。
- **CL-bench Life** [9] 提供群聊、个人档案和行为记录等日常 context；只抽取可验证的个人事实、关系和偏好，避免从 CL-bench 的专业规则与长 procedure 中做高成本能力标注。
- **BFCL Multi-Turn** [14] 提供文件、车辆、交易和旅行等工具 schema，以及 missing-parameter、long-context 和 self-correction 任务；可将默认账号、目录、联系人和 timezone 作为跨 episode 持久事实。
- **轻量合成流是 Core 主体。**用版本化 user profile 和 calendar/file/message mock tools 直接生成 gold state、纠正事件与调用参数；RealUserSim [15] 只提供表达和披露风格。

Core 规模与资源估计

5 个独立 user-lifetime streams × 120 episodes = 600 episodes/run。每条流使用一个 JSON/SQLite fact store、少量文本附件和 calendar / file / message mock tools；目标为 < 50 MB、< 3 CPU-minutes（不含 LLM inference）。由于任务轻量，增加长度的主要成本来自模型推理，而不是环境执行。

4 L2：任务级能力学习与同族泛化

L2 测 learner 能否从多轮成功、失败和局部反馈中形成可跨实例复用的任务级能力。该能力可以表现为显式文本 skill，也可以是 memory、harness 或参数中的隐式更新；benchmark 不要求系统一定维护 skill library。每个任务只在评测侧关联隐藏的 capability labels，包括核心 operator / invariant、所需 tools、domain、representation 和 failure mode；同一能力在不同数据、工具名和用户表达下重复出现。

技术路线：task–capability graph + 多类型 sandbox + capability-aware stream sampler

先为每个任务标注一个或多个可复用能力及其依赖关系，再按标签构造 exposure、同族 probe、surface shift、near-negative 和 replay。Learner 看不到 gold labels，只能从轨迹与局部反馈中归纳。最终 verifier 既检查答案或产物，也检查关键中间状态和成本；对于不依赖显式 skill 的数据问答，重点判断是否复用了正确的 schema 对齐、证据连接和时间过滤策略。

4.1 按任务类型组织，而不是预先固定六个 Skill

任务类型	任务流形式	可复用能力	主要 Verifier
连续数据问答	多轮到达的 CSV/SQLite/日志；schema、实体和时间窗口持续变化	schema 对齐、跨表证据连接、时间过滤、数值校验	答案、evidence rows、query trace
符号与算法推理	新的局部算子、约束系统、定理组合或程序化 puzzle；实例共享 latent rule	归纳不变量、选择规则、分解子目标、反例修正	exact answer、proof/program checker
长程交互任务	需要 15–40 次工具调用的预订、调查、游戏或故障恢复；中途出现失败和状态变化	计划分解、checkpoint、状态追踪、恢复与重规划	环境终态、关键里程碑、side effects
代码与复杂产物	软件修复、配置迁移、表格/文档/媒体交付；输入和交付约束跨实例变化	可执行 procedure、工具链组合、回归检查与产物质量控制	unit tests、artifact tests、资源成本

表 3 L2 按任务类型覆盖数据、推理、长程交互与真实产物，显式 skill 只是能力载体之一。

每条 16-episode Core 流包含 4 个带局部反馈的 exposure、4 个同族 probe、2 个 surface-shift、2 个组合或长程 probe、2 个 near-negative，以及 2 个旧能力 replay / 无关任务。连续数据问答虽不要求调用预置 skill，但后续问题必须复用先前学到的局部 schema、实体映射或查询策略；若每题都能独立求解，则不能作为 L2 主任务。

为避免前沿模型首轮直接饱和，候选 family 必须通过三项校准：无持久更新的 frontier baseline 成功率位于 20%–70%；oracle demonstration / procedure 至少提升 20pp；probe 更换实体、数据和语言表面，且答案无法由预训练知识猜出。难度来自任务内新 specification、长依赖和可验证状态，而不是故意含糊的题面。

4.2 可从哪些 benchmark / 数据中提取

SkillLearnBench [3] 与 SRA-Bench [4] 提供同族实例、数学/逻辑/代码能力标签和 hard negatives；SkillsBench [5] 提供软件、办公、科学与工业产物及确定性 verifier；AppWorld [11]、τ-bench [12] 和 BFCL Multi-Turn [14] 提供长程工具交互与状态检查。连续数据问答优先由受控 relational/temporal generator 合成：先采样查询 program 和 evidence graph，再渲染为 CSV、SQLite、日志或业务报表，从生成程序直接得到答案与证据真值。

Core 规模与资源估计

4 个任务类型、每类 2 个 family，合计 8 families × 3 stream seeds × 16 episodes = 384 episodes/run。推理和数据 QA 使用轻量执行器；长程与产物任务使用 CPU-only 容器，约 0.2–2 GB 冻结依赖、目标 5–30 CPU-minutes。开放 Web 与高成本多模态任务只进入 Stress split。

5 L3：组合泛化与跨任务族结构迁移

L3 测 learner 能否把多个 domain-specific 更新上卷为抽象结构，并在新工具空间中重新实例化；它不把“换实体名”称为举一反三。Source 与 target 在 domain、词汇、representation、tools、action space 和 verifier 上均不同，只共享可声明的 latent structure / invariant。能力标签因此拆成结构标签与表面标签，并构造三种边：structure 高而 surface 低的 Related、难度匹配但 structure 低的 Unrelated，以及 surface 高而 structure 低的 Hard Negative。

5.1 三个可执行 bridge

Source → Target	共享结构	表面变化
FestivalPlan → BuildFix	partial order、cycle detection、min-cost repair	活动 JSON → package manifests / build tools
WarehouseTransfer → MultiServiceBooking	compensating transaction、reverse undo、state conservation	库存工具 → 航班/酒店/支付 APIs
RuleSynth → Property-CodeRepair	counterexample-guided refinement、failure-class induction、regression	符号 grammar → Python / property checker

表 4 L3 的 source 与 target 共享算法结构，但不共享任务表面。

每个 target seed 从同一 snapshot 克隆三臂：Related-Persist、Related-Reset 和 Unrelated-Persist；三臂看到相同数量的 source 任务、反馈和更新机会，并在完全相同的 target probes 上评分。主指标为 Persistent Gain 与 Related 相对 Unrelated 的 Cross-Family Transfer；Hard Negative 单独报告错误迁移率。这样能排除一般练习、额外 token 和 target pre-test 的影响。

5.2 可从哪些 benchmark / 数据中提取

TaskBench [7] 的 tool nodes / dependency DAG 最适合提取原子工具与可控组合，但必须将抽象工具落地为可执行环境；AgentSkillOS [6] 提供多 skill 集合和执行 DAG，可产生“原子均见、组合未见”切分；SRA-Bench [4] 的 CHAMP / BigCodeBench 多 skill 标注适合组合标签；SkillsBench [5] 提供真实交付物和 verifier；CFQ [10] 的 compound divergence 思路可借来控制组合切分。跨域 bridge 的结构真值仍优先由同一个 latent generator 连接两个独立 renderer / verifier，以避免人工类比过强。

Level	Core	每条流资源（工程预算）	12 周 owner
L1	5 streams / 600 episodes	< 50 MB；< 3 CPU-min	Runtime + fact/user simulator
L2	24 streams / 384 episodes	0.2–2 GB；5–30 CPU-min	Labels + sandbox importers
L3	18 arms / 216 episodes	0.5–2 GB；10–40 CPU-min	Bridges + causal analysis
合计	**47 streams / 1,200 episodes**	**3 人 3 个月可完成；第 2/6 周设置 go/no-go。**	

表 5 首版规模。资源估计不含模型推理 token / API 成本。

核心主张

新颖性在于用同一能力图评测 learner 从轨迹中形成的三种更新粒度，并以 episode 内多轮交互、episode 间更新、混合任务与克隆对照模拟真实部署。若 oracle 无法证明 Related > Unrelated，L3 只作为 exploratory track。

Appendix 目录

A. 构造过程中可使用的 benchmark / 数据源

A.1 CL-bench / CL-bench Life
A.2 AppWorld
A.3 τ-bench
A.4 BFCL Multi-Turn
A.5 RealUserSim
A.6 SkillLearnBench
A.7 SRA-Bench
A.8 SkillsBench
A.9 AgentSkillOS
A.10 TaskBench
A.11 CFQ

B. 主要竞品：自进化 benchmark / 评测协议

B.1 竞品总览
B.2 Streaming / lifelong
B.3 Memory evolution
B.4 Harness evolution
B.5 受控反思与工程演化
B.6 我们的差异化边界

C. 实施与评测补充

C.1 基线、指标与数据隔离
C.2 12 周计划与 go/no-go
参考文献

解释边界

A 部分按 benchmark 逐篇介绍原始问题、数据构造、模型运行方式、评测指标和代表样例，不再为每项工作指定某个 Level。正文只负责说明 Tri-EvoBench 的技术路线；Appendix 保持为可独立阅读的相关工作与数据说明。

A 构造过程中可使用的 benchmark / 数据源

本节只描述各 benchmark 本身：它解决什么评测缺口、数据如何构造、模型实际接收什么输入并产生什么输出，以及结果怎样评分。它们与 Tri-EvoBench 三个 Level 的关系只在正文讨论。

A.1 CL-bench / CL-bench Life：学习上下文中的新知识

发表时间：CL-bench：2026 年 2 月 3 日（arXiv v1）；CL-bench Life：2026 年 4 月 29 日（arXiv v1）。

动机、规模与构造

CL-bench [8] 区分“使用预训练知识推理”和“从当前 context 习得新 specification 后再推理”。数据由领域专家编写，共有 500 个复杂 context、1,899 个 task 和 31,607 条二元 verification rubrics；每个 context 平均关联 3.8 个任务，平均约 10.4K tokens，最长约 65K tokens。作者使用虚构知识、系统修改后的规则或长尾/新兴材料降低预训练记忆的影响，并要求解题所需的新知识完整存在于 context 中。51.1% 的任务属于顺序式多轮任务：后续题可能依赖同一 context 下先前任务的结果。

四类学习对象分别是 Domain Knowledge Reasoning、Rule System Application、Procedural Task Execution 与 Empirical Discovery & Simulation，共 18 个子类。四类规模依次为 190/663/11,099、140/566/8,286、100/471/9,486 和 70/199/2,736，三个数字分别表示 context、task 与 rubric 数。专家先写 context，再写只能借助该 context 解决的问题，最后把正确答案拆成事实、计算、规则、异常处理、完整性和格式等细粒度判据。

模型一次接收完整 context、当前 query 以及顺序任务所需的历史，直接生成自由文本答案；原协议不允许外部 skill 检索或参数更新。GPT-5.1 verifier 逐条判断 rubrics，只有全部通过才算整题成功。十个前沿模型平均成功率为 17.2%，最佳 GPT-5.1 为 23.7%，说明困难不只是从长文中检索一句话，而是把局部规则转化为可执行解法。

代表 Case：虚构的 Drone Fleet SDK

类别与 Context：Procedural Task Execution。System context 是虚构的 SkyNet Logistics Drone Fleet SDK v4.5.2，规定 authentication、50 requests/s 限流，以及 Navigation、Payload、Safety 三组 API；Class-4 Hazmat 起飞前必须完成天气、禁飞区、空域许可和货物锁定检查。

用户请求：使用无人机 D-998 向 Sector 4 紧急运送 12kg medical isotopes；当前阵风过强，用户要求调用文档中不存在的 `force_launch_override()` 绕过检查。

模型应做什么：识别函数不存在并拒绝绕过安全协议，但不能只给拒绝；还要仅使用文档中的合法 API，给出包含 weather check、geo/no-fly check、`Safety_request_airspace()` 和 cargo lock 的替代流程，并绑定 D-998、Sector 4 与 Hazmat 参数。

Verifier：多条 yes/no rubrics 分别检查“不存在的函数”“不得 bypass”“合法替代流程”“实例参数与安全步骤”。任何一条失败，整题均记为未解决。

CL-bench Life [9] 将同一问题推进到现实生活 context：群聊记录、个人档案和行为痕迹往往杂乱、碎片化且带有社会关系。该版本完全由人工构造，包含 405 个 context–task 对和 5,348 条 rubrics。十个前沿模型平均成功率 13.8%，最佳为 19.3%；主要失败来自跨片段关系整合和从日常行为中恢复隐含事实，而不是单段文本问答。

CL-bench Life 代表性任务模式：从群聊碎片恢复聚会约束

输入：一段多人群聊、零散日程与个人行为记录；其中昵称、到达时间、饮食偏好和地点变更分别出现在不同消息中，且早期消息可能已被后续更新覆盖。

用户请求：整理最终可执行的聚会方案，并说明时间、参加者、地点与饮食安排。

模型应做什么：解析昵称与人物对应关系，使用最新时间而不是旧消息，把不同来源中的饮食限制和可用时段连接起来，并在答案中显式处理仍未确定的信息。

Verifier：rubrics 独立检查人物映射、最新有效时间、地点、饮食约束和不确定性表达。该案例的难点是跨碎片更新与社会关系推断，而非在单段文字中查找答案。

A.2 AppWorld：可控应用世界中的交互式代码 Agent

发表时间：2024 年 7 月 26 日（arXiv v1；ACL 2024）。

环境与任务构造

AppWorld [11] 针对传统 tool-use benchmark 只需短 API 序列、无法测试复杂控制流的问题，构造了一个可执行的日常数字世界。AppWorld Engine 约 60K 行代码，包含 9 个应用、457 个 API，并模拟约 100 位虚构用户的邮件、联系人、购物、笔记、支付等数字活动；Benchmark 另有约 40K 行代码和 750 个自然任务。每个任务从一个预置数据库 snapshot 开始，Agent 可查询 API 文档、执行 Python 代码并根据返回值继续迭代。

任务通常跨多个应用，要求搜索、过滤、聚合、分支判断和更新状态，而不是复现一条唯一 action trace。评测采用 state-based unit tests：测试最终数据库是否达到目标，同时检查不相关记录是否被意外修改，从而允许多种正确程序但惩罚 collateral damage。原论文中 GPT-4o 在 normal split 上约为 49%，在 challenge split 上约为 30%。

代表性任务模式：根据家庭信息完成跨应用采购

初始环境：Notes 中保存家庭成员的饮食禁忌和本周清单；Contacts/Messaging 中保存家庭群聊；Shopping App 中有商品、价格和库存；支付账户与其他历史订单已预置。

用户请求：“按照家庭清单和饮食限制购买本周杂货，总价不要超过预算，并把最终订单摘要发到家庭群聊。”

Agent 轨迹：查询 Notes 恢复清单与禁忌，解析 Contacts 中的群聊 ID，循环调用商品搜索并处理缺货替代，计算总价后创建订单，最后发送包含商品和金额的消息。Agent 可以写 Python 完成过滤和聚合，因此不存在唯一 gold action sequence。

Verifier：state-based tests 检查订单项目、数量、预算和支付状态，检查消息是否发给正确群聊且内容完整，同时比较数据库 diff，确保没有修改无关订单、联系人或其他用户状态。

AppWorld 的用户请求在任务开始时给出，交互主体主要是 Agent 与 API 世界；它不是持续对话式 user simulator。它的核心价值在于真实 side effects、可复现数据库和强终态 verifier。

A.3 τ-bench：Tool–Agent–User 的动态对话

发表时间：2024 年 6 月 17 日（arXiv v1）。

问题设置与运行方式

τ-bench [12] 关注两个经常被静态函数调用数据忽略的能力：Agent 是否能在动态对话中向用户补齐信息，以及是否遵守领域 policy。每个任务包含初始数据库、用户的私有目标、可用 API、领域规则和目标数据库状态。LLM user simulator 与 tool-calling Agent 轮流发言；Agent 可以澄清、读取数据库、提出行动并在必要时请求确认，用户 simulator 则根据私有目标回答，直到任务完成或达到轮次上限。

原始版本覆盖 retail 与 airline 两个真实服务域。Retail 包括账户认证、订单查询、退换货、地址与支付方式等；Airline 包括订票、改签、取消、行李和旅客信息。评分不比较一条 gold transcript，而是把最终数据库状态与人工标注的 goal state 对齐，并检查必须传达给用户的信息。论文还提出 pass^k：同一任务独立运行 k 次且全部成功才记为可靠，用来揭示单次成功率掩盖的不一致；原实验中 GPT-4o 单次成功率仍低于 50%，retail 的 pass^8 低于 25%。

代表性任务模式：修改零售订单中的商品规格

初始状态与私有目标：数据库中存在多个相似订单；用户希望把最近一笔仍可编辑订单中的蓝色 T-shirt 从 M 码改成 L 码，但初始话语只说“把昨天那件蓝色上衣换成大一码”。

交互要求：Agent 先按 policy 要求 email，或姓名加邮编完成认证；随后查询近期订单与 line-item ID，确认“昨天”“蓝色上衣”对应哪一项，检查订单状态与可修改条件，并向用户复述具体变更。在用户显式确认前不得执行写操作。

工具执行：认证、订单检索和商品定位可有不同顺序，但最终更新调用必须指向正确 order/item，并只改变 size 字段。

Verifier：比较最终数据库与 annotated goal state，并检查 Agent 是否传达确认信息。猜测身份、跳过确认、修改错误订单或产生额外 side effect 都会失败；pass^k 还要求重复运行保持一致。

当前维护仓库在后续版本中继续修订 airline / retail，并扩展 banking、telecom dual-control 和 voice 等环境；这些版本改变了任务与 policy，使用时必须冻结具体 commit，不能把不同版本结果直接混报 [13]。

A.4 BFCL Multi-Turn：多轮、多步骤函数调用

发表时间：2024 年 9 月 19 日（BFCL V3 Multi-Turn 首次发布；此后持续更新）。

API 世界与数据构造

Berkeley Function Calling Leaderboard（BFCL）从单轮 AST 匹配逐步扩展到多轮、多步骤工具交互 [14]。BFCL V3 Multi-Turn 先人工构造八组可执行 API：Vehicle Control、Trading Bots、Travel Booking、Gorilla File System，以及 Message、Twitter、Ticket 和 Math companion APIs；再把工具间的输入输出依赖建成图，从图中采样 execution paths，由人工编写和核验多轮用户请求、初始 state 与最小可行调用轨迹。官方 scored set 包含 200 个 Base、200 个 Missing Parameter、200 个 Missing Function 和 200 个 Long-Context cases。

Base 要求从用户请求、历史工具返回或探索性调用中恢复所有必要信息；Missing Parameter 要求模型发现无法推断的参数并向用户澄清；Missing Function 先隐藏必要函数，模型指出能力缺口后才在后续轮提供；Long-Context 在数百文件或数千记录中加入大量干扰。每个 user turn 内，Agent 可连续调用工具，调用结果写回历史；无有效调用时结束当前 turn，单 turn 超过 20 steps 则强制失败。

评测在每一轮结束后同时运行 state-based 和 response-based checks。写操作比较 backend state，例如文件是否创建、watchlist 是否修改；只读任务则检查必要调用是否包含在模型轨迹中。只有所有轮次均通过才算整条 entry 正确，因此模型不能靠最终一句自然语言掩盖中间的错误 side effect。

代表 Case：BFCL Vehicle Control 按需加油

Backend state：油箱当前有 5 gallons、最大容量 50 gallons；到达 Rivermist 需要额外 44 gallons。
用户请求：“Fill the fuel tank until we are able to reach Rivermist. Oil costs money, so I just need to reach there; I do not need a full tank.”
最小可行调用：`displayCarStatus("fuel")` 读取当前油量，再调用 `fillFuelTank(fuelAmount=44)`。常见错误是直接传入 50，既忽略现有 5 gallons，也违反“不要加满”的偏好并触发容量错误。
Verifier：response-based check 要求必要调用及参数出现；state-based check 验证最终油量足够且未超过容量。所有 user turns 的两类检查均通过，整条 multi-turn entry 才成功。

A.5 RealUserSim：由真实对话行为约束用户模拟

发表时间：2026 年 4 月 7 日（arXiv v1）。

为什么需要 grounded user simulator

RealUserSim [15] 指出默认 LLM 用户往往过度正式、配合度过高，手写“犹豫、简短、会误解”等指令又会被模型夸张执行。前者形成 Formalism Ceiling，后者形成 Directive Amplification，两者都会改变被测 Agent 的难度。作者从 14,000+ 条 WildChat 真实人类–模型对话中抽取 7,275 个可执行 behavioral profiles，用真实行为证据约束模拟，而不是凭研究者想象写 persona。

每个 profile 描述五个行为维度，例如表达形式、信息披露方式、互动配合、修正/反驳和任务推进习惯；生成阶段把 profile 与任务目标同时提供给 simulator。PT3 fidelity benchmark 包含 600 段对话、覆盖 71+ 个领域，并使用 anti-leakage 控制防止模型复述原对话。Grounded simulation 将五维行为 match rate 从 24.2% 提高到 45.3%。

论文进一步把六种 simulator 模型接入 TauBench。相对于 cooperative simulator，grounded profiles 使 Agent 平均 task success 下降约 3.2–3.5 个百分点，并暴露“信息一次没说全”“对澄清问题答非所问”“失败后改变表达但不改变目标”等机制。这里下降不是 simulator 越难越好，而是说明用户模型会系统性改变 Agent 排名和失败分布。

代表性 Profile Case：简短披露并延迟纠正

Grounded profile：用户偏好极短回答，不主动给 account detail，只在被问到时提供；当 Agent 复述错误时才明确纠正，而不是提前给完整 specification。
任务状态：TauBench 中的目标订单、身份字段和要修改的 line item 仍由 symbolic state 固定。Simulator 先说“帮我改一下昨天的订单”；Agent 追问后才得到邮箱；若 Agent 把商品理解错，用户随后回复“不是那件，是蓝色的那件”。

两层评测：PT3 使用真实对话标注，在表达形式、信息披露、配合程度、纠正方式和推进习惯五个维度比较 simulator 与人类行为；接入 TauBench 后，任务是否完成仍由数据库 goal state 判断。
案例意义：Profile 只决定“用户怎样说”，不能改变“什么是真的”。因此该 case 能揭示过度 cooperative simulator 会隐藏的澄清与误解恢复问题，同时避免让用户 LLM 充当任务裁判。

A.6 SkillLearnBench：从经验生成可复用 Skill

发表时间：2026 年 4 月 22 日（arXiv v1）。

任务单元与 skill-dependency 过滤

SkillLearnBench [3] 把一个任务定义为自然语言说明、确定性 verifier、人工参考 skill 集合，以及 2–6 个共享同一核心流程但输入不同的实例。当前版本有 20 个任务、100 个实例，覆盖 6 个大类和 15 个子领域；17 个任务改编自 SkillsBench，另补日程规划、海报设计和中文诗歌。每个任务先经过 skill-dependency 检查：无 skill 的固定 Agent 在各实例上运行 10 次，成功率必须不高于 0.5；带人工 skill 的 Agent 则至少成功一次。

六个大类是 Software Engineering、Information Retrieval、Productivity Tools、Data & Analytics、Content & Creative、Utilities & Other。模型只得到任务说明和一个 seed instance，生成候选 skill；随后固定 solving agent 在同族实例上加载该 skill。这样的角色分离使 benchmark 能区分“skill 生成失败”和“solver 没有正确执行 skill”。论文比较 One-Shot、Self Feedback、Teacher Feedback 与 Skill Creator：前两者分别直接总结和根据自我轨迹改写，Teacher Feedback 允许访问人工 skill 的 teacher 给方向性提示，Skill Creator 使用结构化的意图分析、边界调查、specification 写作与检查流程。

评测分三层：skill 文本的 coverage/可执行性/安全性，solver 轨迹与 oracle 轨迹的接近程度及 skill usage，最终 verifier success 与 token 成本。统一 solver 使用固定 Agent 配置，以避免把更强求解模型误当成更强 skill learner。论文报告无 skill 平均 10.17%，人工 skill 74.50%，自动方法约 27.33%–31.08%；外部 feedback 的多轮更新确有帮助，但纯 self-feedback 容易 recursive drift。

代表 Case：`financial-analysis-1`

类别与输入：Data & Analytics → Data Processing。容器中有 `/root/2025-q2` 与 `/root/2025-q3` 两个季度的 SEC 13F 数据。
用户请求：查询 Renaissance Technologies 在 Q3 的 AUM 和持股数；比较 Berkshire Hathaway 从 Q2 到 Q3 增持金额最高的 5 只股票；找出 Q3 对 Palantir 投资市值最高的 3 位基金经理。
Human skill：`fuzzy-name-search` 将不精确基金/公司名对齐 accession number 或 CUSIP；`13f-analyzer` 读取持仓、计算 AUM、季度差分和排名。Skill learner 只从 seed instance 生成候选 skill，固定 solver 再在同族实例上执行。
输出与 Verifier：写入 `/root/answers.json`。四个 JSON keys 必须齐全；AUM 允许 0.1% 相对误差，持股数和 CUSIP 排名精确匹配，基金经理列表按顺序做大小写无关匹配。后续实例更换基金、季度和文件，但核心流程不变。

A.7 SRA-Bench：大规模 Skill 检索、吸收与应用

发表时间：2026 年 4 月 27 日（arXiv v1；2026 年 6 月更新至 v3）。

语料库与标注

SRA-Bench [4] 将 Skill Retrieval Augmentation 拆成检索、是否加载/如何吸收、以及最终任务执行。Benchmark 有 5,400 个 capability-intensive instances 和 636 个手工整理的 gold skills；gold 与 25,626 个网页收集的 distractor 混合，形成 26,262 个候选 skill，gold 只占约 2.4%。Gold skill 不是题目答案，而是包含名称、适用条件、步骤和常见陷阱的通用能力文档。

运行时先用 BM25、TF-IDF、BGE、Contriever 或 LLM reranker 从全库召回候选，再由模型判断是否加载及加载哪些 skill，最后回答题目或生成代码。对照包括无 skill、直接给 gold 的 Oracle、候选全文注入、LLM Selection 和只展示 metadata 后按需展开正文的 Progressive Disclosure。检索用 Recall@K/nDCG@K，最终任务用规则或单元测试；另构造 $N \in \{0, 2, 4, 8\}$ 个 hard negatives 的受控集合，把“没召回”和“召回后不会筛选”分开。

来源	能力类型	实例	gold skill	映射
TheoremQA	定理应用	747	320	单 skill
LogicBench	逻辑模式	760	19	单 skill
ToolQA	工具工作流	1,430	14	单 skill
MedCalc-Bench	医学计算器	1,100	55	单 skill
CHAMP	数学概念	223	89	多 skill
BigCodeBench	软件库/API	1,140	139	多 skill

表 6 SRA-Bench 的六个来源及 task–skill 标注结构。

代表 Case：CHAMP `champ_00002`

输入：“一个凸 n 边形有多少条对角线？请给出关于 n 的表达式。”公开 instance 的 `skill_annotations` 关联两个 gold skill IDs：`champ_016` 与 `champ_030`。
检索与加载：Retriever 从 26,262 个候选 skill 中返回 top-K；模型再决定是否加载以及加载哪一个。Oracle 条件直接提供两个 gold skills，Progressive Disclosure 条件先只显示名称/描述，模型按需展开正文。
执行：Gold skill 是可复用数学概念和步骤，不直接包含该题答案；Agent 组合能力后推导每个顶点连接 $n-3$ 个非相邻顶点，并消除双计数，得到 $n(n-3)/2$。
诊断：Recall/nDCG 判断是否召回两个能力；已召回却未加载属于 incorporation failure；已加载但答案错误属于 execution failure；最终答案由 CHAMP 规则匹配。

A.8 SkillsBench：Curated Skills 是否真正改善真实任务

发表时间：2026 年 2 月 13 日（arXiv v1；benchmark 持续更新）。

任务包与匹配实验

SkillsBench [5] 当前公开版本有 87 个任务、8 个领域，并为每个任务配 curated skill 和确定性 verifier。每个任务是一个自包含容器：`task.md` 给用户要求，Dockerfile 固定依赖，输入资产放在容器内，`skills/` 中保存一个或多个 SKILL.md 及脚本/参考材料，hidden oracle 证明任务可解，test script 检查最终产物。任务 prompt 通常不直接写 skill 名称；运行时只挂载该任务对应的 skill 目录，而非让 Agent 在全局库中搜索。

八个领域为 Software Engineering（16）、Industrial & Physical Systems（14）、Natural Science（14）、Office & White Collar（14）、Finance & Economics（9）、Mathematics & OR（8）、Cybersecurity（7）和 Media & Content Production（5）。同一模型–harness 在三种匹配条件下运行：No Skills、Curated Skills、Self-Generated Skills。每次任务都创建新容器，Agent 用终端和代码处理输入并提交产物，三种条件由相同 verifier 评分。当前主实验覆盖 18 个 model–harness 配置；curated skills 将宏平均 pass rate 从 33.9% 提升到 50.5%，即 $+16.6$ 个百分点，而自动生成 skill 平均尚不能稳定替代人工 skill。

对当前仓库 snapshot 的统计显示，87 个任务均有 curated skill，共形成 232 个 task–skill 绑定、195 个不同 skill 名称；只有 15 个名称被两个以上任务复用，单任务有 1–7 个 skill modules，`xlsx` 与 `pdf` 是最常见的跨任务交集。这说明它原生更接近“每个任务配一个专家包”的边际收益测试，而不是 task 从共享候选库中选择 skill。

代表 Case：`3d-scan-calc`

任务标签：Industrial & Physical Systems；calculation/extraction；binary、3d-model、JSON；terminal/Python；file-format-knowledge 与 mathematical-method。
输入：`/root/scan_data.stl` 是二进制 STL，但每个 triangle record 末尾的 2-byte Attribute Byte Count 被改作 Material ID；`/root/material_density_table.md` 给出密度，扫描文件还含需过滤的碎片。
Agent 过程：加载 STL/3D scan curated skill，解析 triangles 与 Material ID，按连通性找最大部件，用有向三角形计算闭合网格体积，再按 mass = volume × density 得到质量。
输出与 Verifier：写入 `/root/mass_report.json`，包含 `main_part_mass` 和 `material_id`；Material ID

必须精确，质量相对误差不超过 0.1%。No-Skills、Curated-Skills、Self-Generated-Skills 使用相同输入与测试。

A.9 AgentSkillOS：能力树检索与多 Skill DAG 编排

发表时间：2026 年 3 月 2 日（arXiv v1）。

Benchmark 与离线管理

AgentSkillOS [6] 面向 skill ecosystem 扩展后的两个问题：如何在大库中发现相关 skill，以及多个 skill 怎样按依赖关系执行。Benchmark 有 30 个 artifact-rich tasks，数据计算、文档创建、动态视频、视觉设计和 Web 交互五类各 6 个。专家先从 skill marketplace/GitHub 选择高质量 skill，再反向设计用户请求、输入文件和多个交付物，因此每条任务都有显式 skill 集合。

Manage Skills 阶段读取 skill 名称和描述，让 LLM 对当前节点下的 skill 概括共同能力并提出互斥子类，再把每个 skill 分配到子类；对仍过大的子类递归重复，直到叶节点只含少量或单个 skill。树由整个 skill pool 离线构造，所有 query 共用同一棵树。在线 Solve Tasks 从 root 开始根据 query 逐层选择分支，过滤和去重后保留 top-M（论文设 $M = 8$）；随后 LLM 把子任务和选定 skill 组织成 DAG，同层节点可并行，下游读取上游产物。

实验比较 Quality-First、Efficiency-First 和 Simplicity-First 三种编排策略，以及 oracle skill、只检索不编排、flat invocation 与 no-skill。Skill pool 规模从 200、1K 扩到 200K。复杂产物先渲染成页面图、网页截图或视频采样帧，再用双向 LLM pairwise comparison 判断质量；调换答案顺序仍冲突则记平局，最后通过 Bradley–Terry 聚合成统一质量分。论文结论是 tree retrieval 能近似 oracle selection，而相同 skill set 下 DAG orchestration 显著优于 flat invocation。

代表 Case：`data_computation_task1` – Penguin Species Research

输入与请求：`artifacts/penguins.csv` 包含 species、island、sex、bill length/depth、flipper length 和 body mass。用户要求物种描述统计与显著性比较、各物种内部的性别形态差异，以及至少 4 张 publication-quality PNG。

Gold skill 集合：{`pptx`, `docx`, `scientific-visualization`, `data-storytelling`}。前两项约束交付物格式，后两项负责统计图与叙事组织。

编排与执行：系统从 capability tree 检索 skill，构造“数据分析/制图 → 报告与演示”的 DAG；同层分析节点可并行，下游节点复用图表、统计结果和结论。

输出与评测：生成 `report.docx` 和不少于 10 页的 `presentation.pptx`。客观 evaluator 检查两个文件、至少 4 张 PNG、三个企鹅物种和页数；双向 LLM pairwise judge 比较统计正确性、完整性、设计与美观，再用 Bradley–Terry 聚合。

A.10 TaskBench：由 Tool Graph 生成任务自动化数据

发表时间：2023 年 11 月 30 日（arXiv v1；NeurIPS 2024）。

生成过程与结构类别

TaskBench [7] 将工具生态表示为 Tool Graph：节点是工具，边是资源依赖或时间依赖。HuggingFace Tools、Multimedia Tools 与 Daily Life APIs 三张全局图分别含 23/225、40/449、40/1,560 个节点/边。生成器先采样 1–10 个节点的 Node、Chain 或 DAG 子图，再使用 back-instruction 从 gold graph 反向写自然语言 query、任务步骤、工具参数和依赖边，并经过规则、LLM critic 与人工抽检。

原论文数据共有 28,271 条：HuggingFace 12,217、Multimedia 8,904、Daily Life 7,150。模型只看到自然语言 query 和可用工具说明，需要输出任务分解、工具节点、参数和完整 invocation graph。TaskEval 用 ROUGE 检查任务分解，用 node-F1 / edge-F1 检查工具与依赖，用 parameter/value F1 检查参数，并报告整图 exact accuracy。模型在原设置中主要是预测 plan，并不真正执行所有外部服务；因此 gold DAG 能判 planning 是否正确，却不能保证 API 可调用或终态成功。

代表 Case：从风景视频到带旁白的动漫视频并发布

类别与输入：Multimedia Tools × DAG；用户提供 `video.mp4` 和 `script.txt`，要求将风景视频动漫化、加入脚本旁白并发布。

Gold Tool Graph：`Video Captioning(video.mp4)` → `Text Generation` → `Video Generation`；另一条可并行分支为 `Audio Generation(script.txt)`；随后两支在 `Video Editing` 汇合，再进入 `Video Share`。
模型输出：模型只看自然语言 instruction 和工具说明，需要恢复六个 task steps、各节点参数以及依赖边；上游产物通过 `<node-i>` 传给下游。
TaskEval：node-F1 检查工具选择，edge-F1 检查汇合/依赖，parameter/value F1 检查文件与节点引用；只有节点、边和参数全部一致，整图 exact accuracy 才成功。原 benchmark 不执行真实视频服务。

若需要执行环境，必须额外实现工具、固定输入资产和版本，并用终态 verifier 取代纯图匹配。对当前 Multimedia 数据快照做闭包筛选时，若只保留 12 个可由 FFmpeg/Pillow 稳定实现的工具，可得到 783 个无需未知节点的任务，其中 600 个 single、156 个 chain、27 个 DAG；这是本项目对仓库快照的工程测量，不是原论文报告数字。

A.11 CFQ：以 Compound Divergence 测量组合泛化

发表时间：2019 年 12 月 20 日（arXiv v1；ICLR 2020）。

数据与切分思想

CFQ [10] 不是 Agent benchmark，而是从 Freebase schema 和可控 grammar 生成英语问题–SPARQL 查询对的语义解析数据，共 239,357 对、49,320 个自然语言 question patterns 和 34,921 个 query patterns。每条样本可分解为 atoms（关系、实体类型、操作等基本成分）和 compounds（这些成分的特定组合）。随机切分通常让训练与测试共享大量 compounds，因而会高估组合泛化。

作者提出 Maximum Compound Divergence（MCD）切分：在保证 train/test atom divergence 很小、即基本成分覆盖相近的条件下，最大化 compound distribution 的差异。MCD1、MCD2、MCD3 给出不同高差异切分；三类序列到序列模型的准确率随 compound divergence 增大显著下降，并呈强负相关。这一设计把“没有见过原子能力”和“见过原子但没有见过该组合”区分开。

代表 Case：已见关系原子的未见 Join

训练覆盖：训练集分别出现“谁导演了 M0?”和“谁制片了 M1?”，也出现 director 与 writer、producer 与 actor 等其他 compounds，因此 `directed_by`、`produced_by`、实体类型和 conjunction 原子都已见。
测试问题：“Who directed and produced M2?”测试要求把 director 与 producer 第一次连接到同一 movie variable，并根据语义决定两个关系是否共享同一 person variable，而不是独立返回两个集合。
Gold 输出：SPARQL 包含两个关系 triple patterns、共享的 film/entity constraint 和正确变量绑定。模型必须生成完整查询，而不只是预测关系名称。
评分与意义：exact match 判断整个 SPARQL 结构；MCD split 保持 atom divergence 很小、同时增大 compound divergence，因此失败可解释为未见组合泛化，而不是缺少基本关系词。

CFQ 的关键贡献是切分而非执行环境：迁移到 Agent 时，atoms 可对应 tool/skill/operator，compounds 对应 skill 集合或 DAG；评测应控制原子覆盖，并单独报告未见组合。

B 主要竞品：自进化 benchmark 与评测协议

B.1 竞品总览

表 7 主要竞品如何构造，以及与 Tri-EvoBench 的区别。

工作	时间	评测对象	任务流/构造	与我们的区别
StreamBench	2024-06	连续反馈下是否改进	静态任务重排为在线序列	无 L1–L3 阶梯与 user simulator
LifelongAgent Bench	2025-05	技能依赖、迁移与遗忘	按技能重叠组织任务序列	有依赖但不要求跨表面结构迁移
SkillLearnBench	2026-04	持续生成可复用 skill	20 tasks / 100 同族 instances	L2 竞品；缺少 L1、严格 L3 与混合流
ClawArena	2026-04	演化信息环境中的长期 Agent	时间推进、动态网页/信息与长期任务	环境演化强，但潜在能力关系不可控
SEA-Eval	2026-04	重复、变体、干扰中的效率演化	92 条长度 5 的逻辑序列	细测效率，不提供三层能力标签图
AgentStream	2026-08	跨 benchmark 共享持久 state	300 题组织为 isolated / sequential / interleaved	多域流强，但关系主要由 benchmark 域定义
Evo-Memory / EvoMemBench	2025-11 / 2026-05	memory retention、revision、execution	episode 内/间与 cross-environment 设置	聚焦记忆模块，不区分 L2/L3 因果迁移
BenchTrace	2026-05	反思与受控失败迁移	signal + noise + 截断 test snapshot	failure 真值强，但不覆盖事实/技能/结构全阶梯
SEAGym	2026-06	可审计 harness evolution	update-train/validation、held-out、replay	是协议基础设施，不定义学习内容层级
Harness Updating Is Not Harness Benefit	2026-05	evolver 与 solver 解耦	强弱模型交叉生成/使用 harness	诊断角色差异；没有统一任务生成器
Rethinking Harness Evolution / Harness-Bank	2026-07	匹配搜索预算、held-out transfer 与 gated search	parallel / sequential / shared harness 对照	预算控制强；不含交互用户和能力层级
RSIBench-Data	2026-07	数据研究 Agent 的递归改进	研究 Agent 构造数据以改进另一个模型	参数/数据闭环，与部署任务流问题不同
Frontier-Eng	2026-04	单任务 artifact 持续优化	47 个工程任务反复 propose–evaluate	是 task-internal optimization，不天然跨任务学习

B.2 Streaming / lifelong 竞品

StreamBench 与 LifelongAgentBench

StreamBench 的关键贡献是把反馈和历史暴露按顺序组织，使 Agent 在未来实例上体现 continuous improvement；LifelongAgentBench 进一步用技能重叠控制正迁移、负迁移与遗忘。Tri-EvoBench 继承它们的 stream / replay 观念，但将任务关系从粗粒度领域重叠推进到可审计的 fact、operator、structure 标签，并在同一流加入 near-negative 和交互用户。

ClawArena

ClawArena 将评测放进持续变化的信息环境，时间、网页内容和可用证据会演化，因而强调长期调查、状态追踪和适应环境变化。它回答“Agent 能否跟上世界”，Tri-EvoBench 更强调“同一段经验究竟造成了哪一层能力变化”；前者适合做高真实性 Stress track，后者适合做受控因果 Core。

SEA-Eval

SEA-Eval 由 32 个原子任务构造 92 条长度为 5 的 correlated、orthogonal 与 implicit-intent 序列，并比较 clean / noisy memory。它发现成功率相近的系统，token 效率可能相差巨大。Tri-EvoBench 保留 efficiency curve 与 interference 诊断，但把重复/变体扩展为 L1 更新、L2 组合和 L3 跨族三种明确 probe。

AgentStream

AgentStream 从 AppWorld、BFCL、BrowseComp+、HLE、SWE-bench 和 Tau2 各取 50 题，将同一 300 题集合组织为 isolated、sequential 与 interleaved scenario，研究共享状态何时产生跨域干扰。Tri-EvoBench 同样需要混合流，但不把“来自不同 benchmark”自动视为跨域迁移；只有共享 latent structure 且通过 Related / Unrelated 对照的 pair 才进入 L3。

B.3 Memory evolution 竞品

Evo-Memory 与 EvoMemBench

Evo-Memory 将 memory lifecycle 的写入、组织、检索、更新和遗忘作为自进化对象。EvoMemBench 再以 episode 内/跨 episode、知识/执行两个轴重构多种数据，并比较不同 memory 系统。两者是 L1 与 memory baseline 的直接竞品；Tri-EvoBench 的额外主张是同一协议还测 procedure induction 与跨 tool-space 的结构重新实例化，而不是把所有提升都归为记忆质量。

SkillLearnBench

SkillLearnBench 已经直接测量从 seed task / feedback 生成 skill 并用于同族实例，是最接近 L2 的竞品。Tri-EvoBench 的差别不是再生成更多同类实例，而是把 skill 放入混合流，加入选择性激活、错误合并、未见组合、near-negative 和跨族 Related / Unrelated 对照，并向下连接 L1、向上连接 L3。

B.4 Harness evolution 竞品

SEAGym

SEAGym 将 update-train、冻结的 update-validation、held-out ID/OOD、replay、成本和 snapshot 制度化，适合作为 Tri-EvoBench 的运行协议参考。两者的关系更像 infrastructure 与 content：SEAGym 规定更新如何被安全选择和评估；Tri-EvoBench 规定要学习什么、任务之间为何应或不应迁移。

Harness Updating Is Not Harness Benefit

该工作让不同模型分别担任 Evolver 与 Solver，说明写出好 harness 和使用 harness 是两种能力。Tri-EvoBench 将这一分解落实为 Update@Anchor 与 Benefit@Oracle：前者固定强 solver 检查更新质量，后者给 oracle update 检查目标 Agent 的激活/遵循，避免只给一个最终总分。

Rethinking the Evaluation of Harness Evolution 与 HarnessBank

前者强调 matched-budget comparison 和 held-out transfer，防止把多次搜索误称为进化；HarnessBank 进一步强调候选 validity、activation、significance gate 与 sealed test。Tri-EvoBench 采用同预算搜索、验证集只用于接受/拒绝、最终 hidden seeds 一次评测，但把门控结果按 L1/L2/L3 和错误类型拆开报告。

B.5 受控反思与工程演化

BenchTrace

BenchTrace 收集带 ground-truth failure mode 的失败 snapshot，并构造 signal–noise–test 流，独立评测失败检测、定位、诊断和 failure avoidance。它是 L2 failure-mode 标签与长距离噪声实验的重要来源。Tri-EvoBench 进一步要求反思产物形成可调用 skill，并在不同实例、组合与跨域 target 上产生可验证收益。

Frontier-Eng

Frontier-Eng 把 47 个工程任务表示为只读规范、可编辑初始 artifact 与冻结 evaluator，让 Agent 在固定预算内持续优化可行分数。它很好地测量单任务内部的 generative optimization，但默认历史服务于同一个 artifact；Tri-EvoBench 只有在该搜索策略或经验跨任务持久化并通过 held-out probe 时，才将其计为自进化。

RSIBench-Data

RSIBench-Data 关注研究 Agent 能否设计、过滤和组织训练数据，进而改进另一个模型，属于数据/参数层的递归改进闭环。Tri-EvoBench 首版主要是部署期 Context / Harness track；后续 Parameter track 可复用同一 L1–L3 任务流和 replay 测试，但需额外匹配训练 FLOPs 与数据可见性。

B.6 我们的差异化边界

- **不是更长的 stream：**新意在对 evolution learner 更新抽象层次的层级评测和克隆对照，而非 episode 数量。
- **不是把 gold skill 发给模型：**gold 只用于数据切分、oracle 与诊断，主赛道必须从轨迹中学习。
- **不是泛化到任意开放世界：**L3 只声称受控任务族间的结构迁移。
- **不是只测记忆：**必须同时报告更新质量、选择/激活、执行遵循和副作用。
- **不是只测自然语言对话：**用户交互提供信息，但成功由工具状态和 artifact verifier 决定。

C 实施与评测补充

C.1 基线、指标与数据隔离

项目	最低要求
状态对照	Reset/Frozen、Raw-Replay、Evolving、Matched-Search、Oracle-Update
能力结果	L1 fact/rule、L2 ID/surface/composition、L3 Related/Unrelated/Hard-Negative 分开
持续学习	forward transfer、replay regression、stale error、skill intrusion、forgetting
机制诊断	update precision、skill activation、procedure adherence、state diff、证据 episode
效率	input/output token、rollout、tool call、verifier call、wall-clock、memory / skill size
可靠性	多 stream order、stream-level bootstrap CI、最坏流、collapse / rollback 次数
隔离	update-train 可见反馈；update-validation 只做 gate；held-out probes 不写回；hidden seeds 最终一次运行

表 8 主 leaderboard 不给单一分数，按 Level 和成本报告 Pareto frontier。

主要增益定义为

$$\Delta_{\text{evo}} = R_{\text{evolving}} - R_{\text{reset}}, \qquad \text{XFT} = R_{\text{related,persist}} - R_{\text{unrelated,persist}}.$$

同时报告 headroom-normalized gain 与 matched-search residual。统计单位是独立 stream / bridge seed，而不是把同一生成器产生的几百条相关 episode 当作独立样本。

C.2 12 周计划与 go/no-go

表 9 3 人 3 个月实施计划。

周	工程目标	研究校准	交付/决策
1–2	统一 runtime、state API、user simulator；30 题纵向切片	3 个模型的 floor / ceiling；oracle learnability	第一次 go/no-go
3–4	L1 两域、L2 三个 task families、Reset / Replay / Evolving	区分记忆、额外搜索与真正更新	Smoke v0.1
5–6	L1 四域、L2 八个 task families、首个 L3 bridge	Related / Unrelated / Hard-Negative oracle test	第二次 go/no-go
7–8	完成三个 bridge；导入 benchmark substrates	composition / surface holdout；人工标签审计	Core v0.5
9	批量生成、verifier fuzzing、容器冻结	4–5 个模型 pilot，清除 floor / ceiling	数据 freeze
10	全 baseline、成本与状态日志	多顺序、Update / Benefit 分解	主表与消融
11–12	Release、文档、leaderboard schema	bootstrap、case study、10% 人审	v1 + paper

解释边界

第 2 周要求 L1/L2 oracle update 至少提升 15pp，且 reset success 位于 20%–80%；第 6 周要求至少两个 L3 bridge 的 Related 相对 Unrelated 提升超过 8pp，且 Hard Negative 不产生同方向收益；第 8 周 verifier 确定性超过 99.5%。任何门槛未通过都先收缩主张或修任务，不继续堆数据规模。

参考文献

[1] Cheng-Kuang Wu et al. *StreamBench: Towards Benchmarking Continuous Improvement of Language Agents*. 2024. https://arxiv.org/abs/2406.08747

[2] Junhao Zheng et al. *LifelongAgentBench: Evaluating LLM Agents as Lifelong Learners*. 2025. https://arxiv.org/abs/2505.11942

[3] Shanshan Zhong et al. *SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks*. 2026. https://arxiv.org/abs/2604.20087

[4] Weihang Su et al. *Skill Retrieval Augmentation for Agentic AI*. 2026. https://arxiv.org/abs/2604.24594

[5] Xiangyi Li et al. *SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks*. 2026. https://arxiv.org/abs/2602.12670

[6] Hao Li et al. *Organizing, Orchestrating, and Benchmarking Agent Skills at Ecosystem Scale*. 2026. https://arxiv.org/abs/2603.02176

[7] Yongliang Shen et al. *TaskBench: Benchmarking Large Language Models for Task Automation*. 2023/2024. https://arxiv.org/abs/2311.18760

[8] Shihan Dou et al. *CL-bench: A Benchmark for Context Learning*. 2026. https://arxiv.org/abs/2602.03587

[9] Shihan Dou et al. *CL-bench Life: Can Language Models Learn from Real-Life Context?* 2026. https://arxiv.org/abs/2604.27043

[10] Daniel Keysers et al. *Measuring Compositional Generalization: A Comprehensive Method on Realistic Data*. 2019. https://arxiv.org/abs/1912.09713

[11] Harsh Trivedi et al. *AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents*. 2024. https://arxiv.org/abs/2407.18901

[12] Shunyu Yao et al. *τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains*. 2024. https://arxiv.org/abs/2406.12045

[13] Sierra Research. *τ-bench / τ^2-bench maintained environments and releases*. 2026. https://github.com/sierra-research/tau2-bench

[14] UC Berkeley Gorilla Team. *BFCL V3: Multi-Turn and Multi-Step Function Calling*. 2024–2026. Official project page.

[15] Salesforce Research. *RealUserSim: Bridging the Reality Gap in Agent Benchmarking via Grounded User Simulation*. 2026. https://arxiv.org/abs/2605.20204

[16] Haonian Ji et al. *ClawArena: Benchmarking AI Agents in Evolving Information Environments*. 2026. https://arxiv.org/abs/2604.04202

[17] Sihang Jiang et al. *SEA-Eval: A Benchmark for Evaluating Self-Evolving Agents Beyond Episodic Assessment*. 2026. https://arxiv.org/abs/2604.08988

[18] Dong Yan et al. *AgentStream: How Well Do Self-Evolving LLM Agents Perform Under Streaming Tasks?* 2026. https://arxiv.org/abs/2608.00155

[19] Tianxin Wei et al. *Evo-Memory: Benchmarking LLM Agent Test-time Learning with Self-Evolving Memory*. 2025. https://arxiv.org/abs/2511.20857

[20] Yuyao Wang et al. *EvoMemBench: Benchmarking Agent Memory from a Self-Evolving Perspective*. 2026. https://arxiv.org/abs/2605.18421

[21] Jiahao Huang et al. *BenchTrace: A Benchmark for Testing Reflection Ability and Controlled Evolution in LLM Agents*. 2026. https://arxiv.org/abs/2605.29225

[22] Congjie Zheng et al. *SEAGym: An Evaluation Environment for Self-Evolving LLM Agents*. 2026. https://arxiv.org/abs/2606.17546

[23] Minhua Lin et al. *Harness Updating Is Not Harness Benefit: Disentangling Evolution Capabilities in Self-Evolving LLM Agents*. 2026. https://arxiv.org/abs/2605.30621

[24] Yike Wang et al. *Rethinking the Evaluation of Harness Evolution for Agents*. 2026. https://arxiv.org/abs/2607.12227

[25] Xiaotian Luo et al. *HarnessBank: Semantic Gene-Bank Search with Gated Verification for Agent-Harness Self-Evolution*. 2026. https://arxiv.org/abs/2607.13683

[26] *RSIBench-Data: Benchmarking Data-Centric Research for Recursive Self-Improvement*. 2026. https://arxiv.org/abs/2607.25886

[27] *Frontier-Eng: Benchmarking Self-Evolving Agents on Real-World Engineering Tasks with Generative Optimization*. 2026. https://arxiv.org/abs/2604.12290